Snake Game

1. Introduction

The Snake Game project implements the classic game using the Java Swing framework. The solution is designed with a strong object-oriented structure to manage core game logic, rendering, and data handling separately.

The key features of the application include:

- Player Persistence: The system maintains player data, including high scores, saved to a file for use across multiple sessions.
- Multiple Food Types: Introducing varied gameplay mechanics through Normal, Golden, and Poison food items.
- Save/Load Functionality: A robust system that allows players to save and resume their exact game progress at any time.

2. Program Interface Prerequisites and Setup

The program requires the Java Development Kit (JDK) version 11 or higher to be installed and correctly configured on the operating system.

Running the Program :

The program does not accept command-line parameters. It is executed from the command line after compilation using the class >> java Main

Program Termination

The program can be terminated in two primary ways, both of which trigger an auto-save procedure before exit:

- External Exit: Clicking the standard window close button (X). This triggers a WindowListener which automatically performs a high score save and prompts for game save confirmation before calling System.exit(0).
- Menu Exit: Selecting the File → Exit menu option, which prompts the user to confirm saving data before calling System.exit(0).

3. Program Execution

1: Starting the Game Flow

1. Player Identification: Upon launch, the user is prompted to Enter your Player ID (must be numeric otherwise the player will be asked again)
2. Name Input: If the ID is new, the user is prompted to enter their Name; otherwise, their existing name and high score are displayed.
3. Load Check: If a saved game is found for the Player ID, the system asks the user if they wish to Continue or Start New.
    - Continue: The game state is loaded, and gameplay resumes.
    - Start New: The old save file is deleted, and a new game starts with a score of zero.

2: Controls and Interface

The game is controlled using standard directional keys, and the game state is managed by the SPACE key.

Key	Action	functionality
W,S,A,D	Movement	Changes the snake's direction. **Reverse movement is prevented** (e.g., cannot move Up while currently moving Down).
SPACE	Pause/Resume	Toggles the game timer and running state.
ENTER	Restart Game	Get activated when the GAME OVER screen is displayed, resetting the board.

Game Over Conditions

The game ends immediately when the snake's head collides with either:

- The Wall (Boundary collision).
- Its Own Body (Self collision).

Upon Game Over, the system automatically attempts to save the score as a high score, and the saved game state is deleted.

4. Input and Output

1: User input format

The application relies on keyboard input for movement and mouse interaction for initial dialogues. The Player ID must be entered as a valid integer.

2: File and Persistence Format

- Player Data File (players.txt)

This file is a **Text File** used for tracking permanent high scores. The system only updates a player's record if the newly achieved score is **strictly greater** than the previously saved high score.

Structure: ID,Name,HighScore,DateTim

- Game Save File (savegame.dat)

This is a **Serialized Binary File** used to store the complete game state. The file contains a map keyed by Player ID and holds the GameState object, which includes the precise coordinates of every snake segment, all food items, current direction, and score.

5: Program Structure

1: Game Loop Initialization

The execution of the game begins immediately after the player data is entered and the load/new game decision is finalized. The Main class initiates the game flow by calling game.start()

1. **Timer Activation:** The game.start() method initiates the javax.swing.Timer, which runs on the Event Dispatch Thread (EDT) and is set to the movedelay value (150ms)
2. **Continuous Execution**: Upon each tick of the timer, the Playingthegame() function is called. This function is executed continuously at the fixed interval, forming the core of the game loop.
3. **Iteration Cycle**: In each iteration, the system executes the sequential logic:

Move Snake > > Check Wall/Self Collisions >> Check Food Collision >> Update Score Display >> Repaint.

2: System Overview and Data Structures

Note: The system's overall structure is defined by the UML Class Diagram below and the following will explain the core code

Core Data structure and collections

structure	Class	purpose
LinkedList<point>	Snake	Used to store the snake's body segments and it's the preferred because during the movement of the snake , the segments will be **shifted automatically** , as they are stored in a Linked list
point	Point	An immutable class defining grid coordinates (x, y). Overriding equals() to be able to compare the position of any food and snake segment and hashCode() ensures reliable and **quick checking for collision**
ArrayList<Food>	Board	Stores all active food instances. Provides simple management for iterating during checks and quick removal upon consumption
Map<Integer, GameState>	SaveloadMnager	Used for persistence, mapping the **playerID** to its **specific Game** where each player will have its **own game** so we can load his own game and save it as well

2: Module Descriptions and Algorithmic Logic

A. Game.java (Controller / Game Loop)

- Role: Controls the entire game state, manages timing via the javax.swing.Timer, and enforces all rules and state transitions.
- Algorithmic Logic:
  - **Game Loop Execution:** The central function, Playingthegame(), is called continuously by the timer in the setupTimer(). It executes the core sequence of operations: move snake, check collisions, check food, and repaint.
  - **Collision Checks:** Both wallcollision() and selfcollision() follow the same critical protocol: check the snake's head position, set GameOver=true if collision is found, stop the timer, and execute the high-score save followed by the game-save deletion protocol.

- **Food Detection**: The foodcollision() method checks if the snake's head coordinates perfectly match any food coordinates. If a match is found, it calls Board.eaatingfood() to execute the consumption and score update.
- **Input Handling**: The key input processing method prevents the user from selecting the movement direction opposite to the current heading (e.g., preventing UP if the snake is moving DOWN), ensuring the snake cannot instantly kill itself.

B. snake.java (Model)

- Role: Defines the entity and manages the internal body structure.
- Algorithmic Logic:
  - **Movement (move()):** A new point object is added to the front of the LinkedList (addFirst), and the oldest segment is removed from the back (removeLast). This simulates movement while maintaining constant length.
  - **Growth (grow()):** Growth is achieved by adding new segments to the tail (addLast) without removing the existing tail segment.
  - **Shrinkage (shrinking()):** Segments are removed from the end of the LinkedList, with a critical check to ensure the length is never allowed to fall **below 1.**

C. Board.java (View / Logic Container)

- Role: Responsible for rendering and managing food generation logic.
- Algorithmic Logic:
  - **Food Generation (Creatingfood()):** This function implements conditional spawning: it enforces that only **one Normalfood** exists at a time, and it calculates and generates golden /poison foods based on score multiples (5 and 10)
  - **Position Validation:** The **isFoodPositionInvalid()** method performs sequential linear checks against all snake segments and all existing food positions to ensure a newly generated food coordinate is unique and available

6. Improvements and Extensions

1: Areas for Improvement and Game Balance

- **Difficulty Levels:** Implement adjustable speed (reducing the Timer delay) based on the player's score to introduce dynamic difficulty.
- **Dynamic Food Timers:** Introduce a strict vanish timer for high-value foods (Golden Food and Poison Food). This forces players to react quickly and improves gameplay balancing.
- **Performance Optimization:** using HashSet<point> for faster lookups and maintaing performance as the sneak grows.

Future Extensions

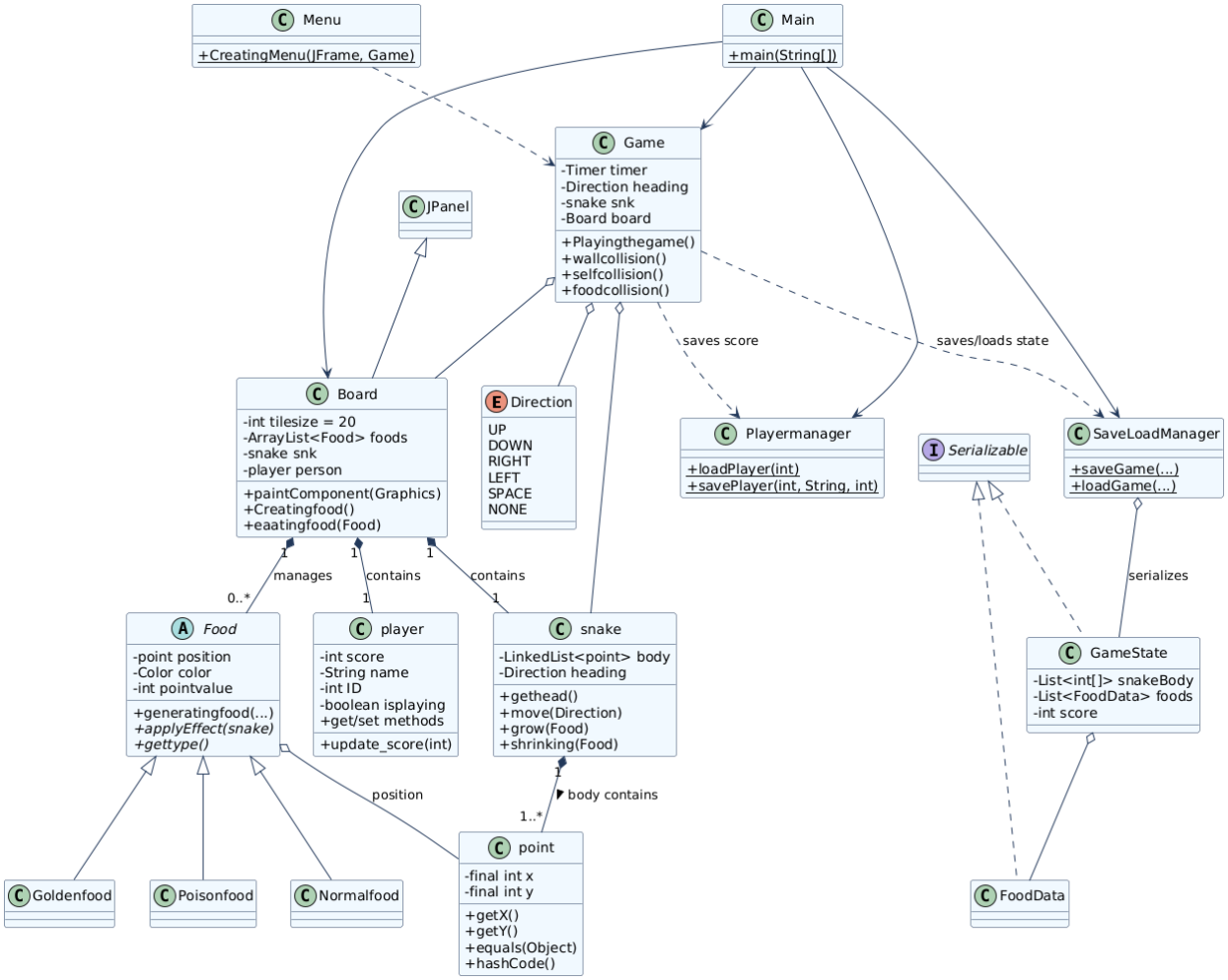
- **Admin Panel / Game Control Interface**: Develop a separate, powerful interface for an "admin" to control game parameters (speed, spawning rates) in real-time**.**
- **UI/UX:** Refine the existing Swing UI for a cleaner, more modern look**.**
- **Audio Feedback:** Add sound effects and background music for a richer user experience.

7: Conclusion

The Snake Game project successfully met its core requirements, delivering a fully functional and extensible implementation of the classic game with robust features. The application utilizes the **Java Swing** framework and **object-oriented principles** to compartmentalize core logic, rendering, and persistence effectively. The successful implementation of the multi-food system, high-score management, and game save/load functionality provides a solid foundation for future development and enhancement.

Diagrams

UML Class Diagram for the entire project



Purpose : The UML Class Diagram is the static blueprint of the software. It defines the structure, showing all major classes, their data fields (attributes), functions (methods), and how they are permanently linked , and also it highlights the usage of the object-oriented design :

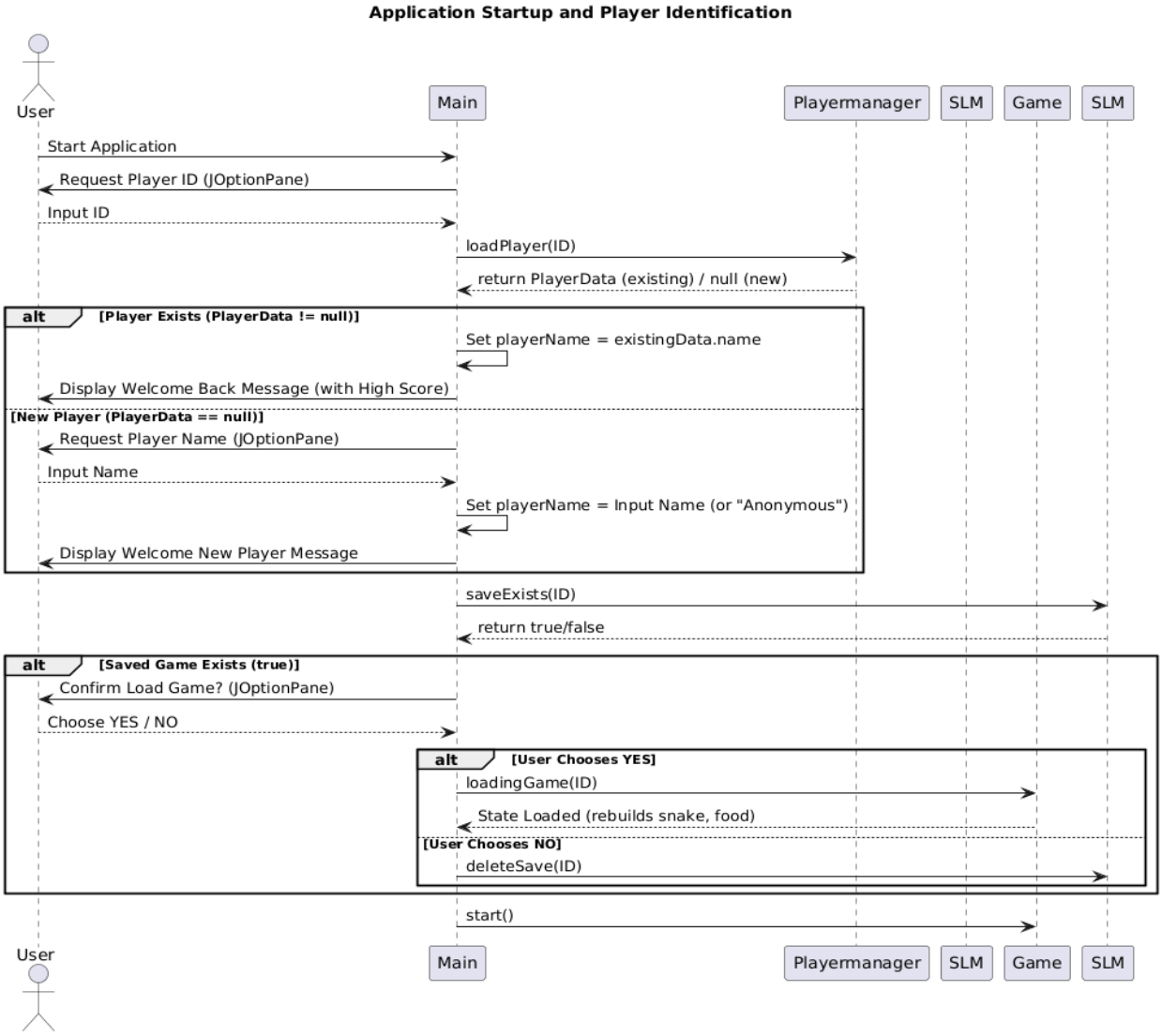
Model/Entity Layer: Classes like snake, player, and point define core data. The snake is structured using a LinkedList<point> for efficient movement.

Polymorphism: The Food system uses inheritance, allowing Board to call a single method (applyEffect and the gettype as well) while subclasses (Normalfood, Poisonfood) execute owned functions > growth or shrinking logic.

Composition: it executes the Has-relationships > such as the Board containing the snake and player objects and the same for some other classes like the has relationship between the Game and the board classes

Persistence: The diagram illustrates the use of the Serializable interface for GameState, enabling the SaveLoadManager to capture the entire game state (snake position, food list, score) for saving and loading , and also it illustrates the input and output of the players Data (name , Id , score , data and time)

Sequence Diagram 1: Application Startup and Player Identification



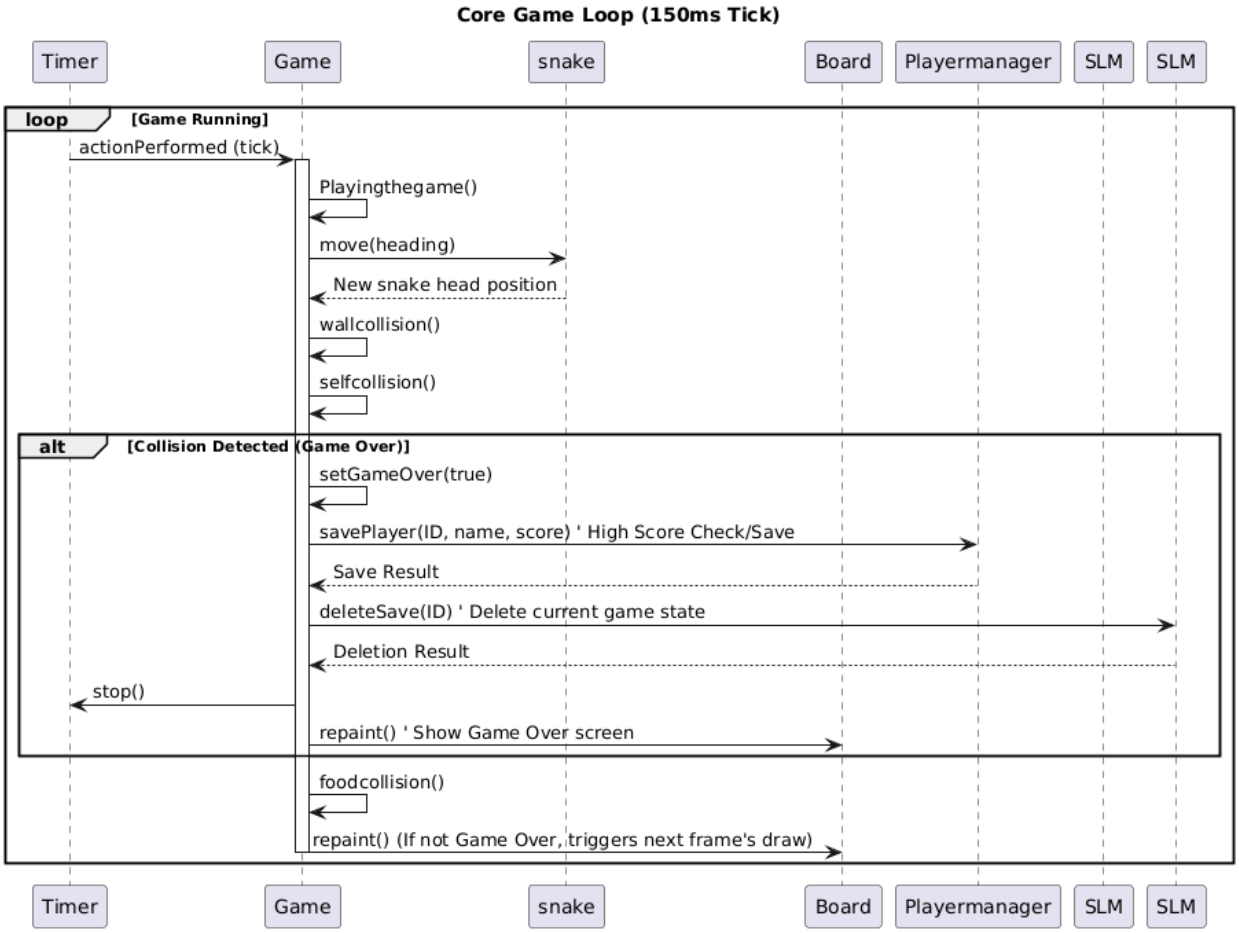
The Diagram is mapping the flow for the launching of the Game

Player Validation: The sequence begins with Main checking the Playermanager to determine if the user is new or existing. The Player Name is conditionally requested only for new users.

Load Decision: After confirming identity, Main checks the SaveLoadManager for any previous game state.

Conditional Loading: The diagram details the nested choice: selecting YES triggers game.loadingGame() to **restore** the save. Selecting NO first executes SaveLoadManager.deleteSave() to remove the old data, ensuring a clean new game starts.

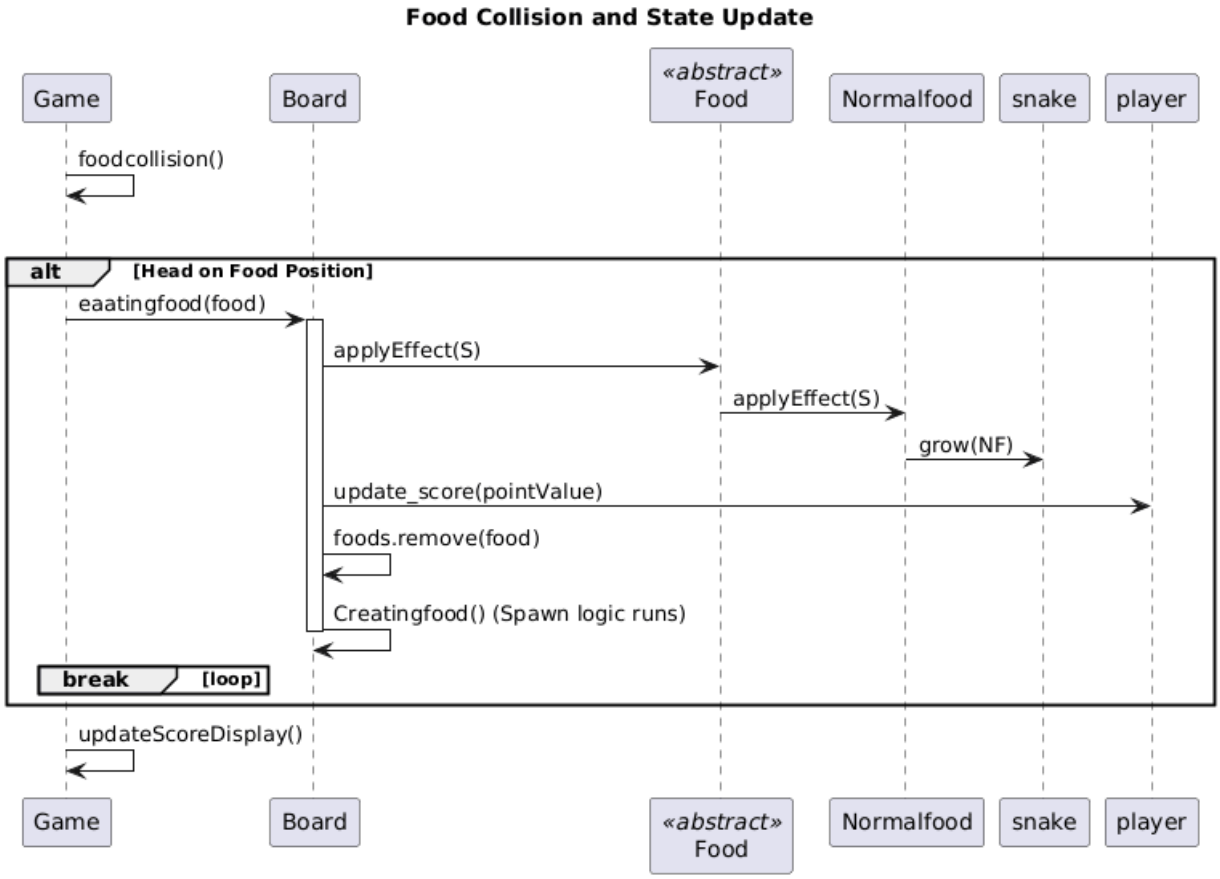
Sequence Diagram 2: Snake Movement and Game Tick



The diagram: represents the Core Game Loop, driven by the Timer every 150ms

- **Game Tick:** The Game Class initiates movement (snake.move()), followed by sequential safety checks: wallcollision() and selfcollision().
- **Game Over:** The critical alt block demonstrates the mandatory sequence for a collision event and the following happens when it reaches the Game over state
  1: set the Gameover state:
  2: check and save the new high score > playermanager
  3: call the function of (deletesave(ID) to remove the current losing Game and stop the timer

Sequence Diagram 3: Food Collision and State Update



The diagram :illustrates the procedure that happens , when any food item is consumed and this proves that the polymorphism and updates functions is working in which

1: the Game will detect the collision and calls the **eatingfood function**()

2: then according to the type of food that has been eaten , we will call its function and it will execute the code which it has different affects on the snake bases on the type of the food

Note on Diagram Generation: The UML Class and Sequence Diagrams were created using the **PlantUML** frendering engine. The necessary code (defining all classes, methods, and relationships) was **generated by an AI model** based on the project's source code, and then rendered to produce the final images.

Unit Test Descriptions (UnitTests.java)

1: player tests

Test's name	Test's explanation
testConstructorSetsScoreAndIsPlaying	Verifies the constructor correctly initializes the score and isplaying state.
testSetName and testsetID	They ensure that both of these setter methods correctly assigns the player's name and their IDS
testUpdateScoreAddsValue	Confirms the update_score method correctly adds a value to the current score.
testSetScoreReplacesValue	Confirms the setScore method correctly overwrites the score with a new value
testSetIsPlaying	Ensures the setIsplaying method correctly updates the game state flag

2:Food tests

Test's name	Test's explanation
testNormalFoodValues	Verifies the color, point value (1), and type ("Normal") are correct upon instantiation.
testPoisonFoodConstructor	Verifies the color and type ("Poison") are correct upon instantiation.
testGoldenFoodConstructor	Verifies the color, point value (3), and type ("Golden") are correct upon instantiation.
testSetAndGetPosition	Ensures the setPosition and getPosition methods accurately store and retrieve the food's grid coordinates (point).

3: Snake tests

Test's name	Test's explanation
testSnakeInitialization	Verifies the snake starts with a length of 1 (the head) and the head object is not null.
testGetHeadReturnsFirstSegment	Ensures gethead() correctly returns the first element of the internal body list.

testMoveUp/Down/Left/Right	Verifies that movement in all four directions correctly updates the head's coordinates (xand y values)
testMoveDoesNotChangeLength	Confirms that a standard movement operation does not inadvertently change the snake's length.
testGrowNormalFood	Verifies growth increases the length by 1 segment (point value 1).
testGrowGoldenFood	Verifies growth increases the length by 3 segments (point value 3).
testGrowPoisonFoodDoesNotGrow	Confirms that poison food does not trigger the growth mechanism.
testShrinkWithPoisonFood	Verifies the shrinking method correctly reduces the snake's length by the poison value.
testShrinkCannotGoBelowOne	Tests the critical edge case that the snake length cannot be reduced below 1 segment (the head).

4:Board tests and intersection with foods

Test's name	Test's explanation
testBoardInitialization	Confirms the dimensions, tile size, and the creation of essential game objects (snake, player, foods).
testBoardStartsWithNormalFood	Asserts that at least one Normalfood is present immediately after board initialization.
testAdd/RemoveFood	Basic checks to ensure foods can be correctly added to and removed from the food list.
testFoodPositionInvalidDueToSnake	Verifies food cannot be placed directly on the snake's head (collision check).
testFoodPositionInvalidDueToOtherFood	Verifies food cannot be placed on the exact location of another existing food item.

testFoodPositionValid	Confirms that a location away from all entities is considered a valid spawn point.
testEatingFoodIncreasesScoreAndRemovesFood	Asserts that consuming food updates the player score and correctly removes the food item from the board's list.
testCreatingNormalFoodIsNotAllowedWhenOneExists	Verifies the spawn guardrail that prevents more than one Normalfood from existing simultaneously.

5: Game Collision & Scoring Tests

Test's name	Test's explanation
testWallCollisionTriggersGameOver	Simulates snake movement to the boundary and verifies wallcollision() correctly sets isGameOver to true.
testSelfCollisionTriggersGameOver	Manually places snake segments in a loop by moving the snake towards one of his segments , forces a self-collision, and verifies selfcollision() sets isGameOver to true
testFoodCollisionEatsFoodAndUpdatesScore	Manually moves the snake's head to a food location and verifies that game.foodcollision() successfully triggers eating and score update.
testEatingPoisonFoodUpdatesScoreCorrectly	Verifies the complex score deduction logic for Poison Food, confirming that the score is accurately reduced by the defined amount (e.g., 8 becomes 4 with a point value of -4)